

Propuesta Arquitectónica – Sistema Jerárquico de Atención al Cliente

1. Identificación de agentes

El sistema utilizará una **topología jerárquica**, donde un **Supervisor Agent** será responsable de coordinar a distintos agentes especialistas.

Los agentes propuestos son:

- **Supervisor Agent**
- **Sentiment Analysis Agent**
- **Order Status Agent**
- **Response Generator Agent**
- **Response Validator Agent**

2. Responsabilidades y herramientas

Supervisor Agent

Es el punto central de coordinación. Recibe el tweet, mantiene el estado del proceso y decide qué agente debe intervenir en cada etapa.

Responsabilidades:

- Recibir y analizar inicialmente la solicitud.
- Delegar tareas a los agentes especialistas.
- Mantener el contexto compartido.
- Gestionar errores o reintentos.
- Autorizar la respuesta final.

Tools:

- LangGraph para gestionar el flujo y el estado.
- LLM para clasificación y toma de decisiones.

Sentiment Analysis Agent

Analiza el contenido del tweet para identificar el sentimiento y la gravedad de la queja.

Por ejemplo, podría clasificar:

sentiment = "negative"
severity = "high"

Tools:

- LLM con structured output.
- Modelo específico de análisis de sentimiento, si fuera necesario.

Order Status Agent

Obtiene información del pedido asociado al cliente.

Responsabilidades:

- Identificar el pedido correspondiente.
- Consultar su estado.
- Recuperar información como fecha de envío, transportista, retrasos o entrega estimada.

Tools:

- API del sistema de pedidos.
- Base de datos SQL.
- API del operador logístico.

Ejemplo de resultado:

```
{  
  "order_id": "ORD-5821",  
  "status": "delayed",  
  "expected_delivery": "2026-08-20"  
}
```

Response Generator Agent

Genera una respuesta personalizada utilizando el tweet original, el sentimiento detectado y el estado real del pedido.

Por ejemplo:

Hola Lucas, lamentamos la demora con tu pedido ORD-5821. Detectamos que el envío sufrió un retraso y actualmente está previsto para el 20 de agosto. Estamos siguiendo el caso.

Tools:

- LLM.
- Prompt con políticas de comunicación de la empresa.
- Templates corporativos.

Response Validator Agent

Revisa la respuesta antes de enviarla al cliente.

Responsabilidades:

- Verificar que no se inventen datos.
- Validar que la respuesta respete las políticas de la empresa.
- Controlar tono y lenguaje.
- Detectar información sensible.
- Aprobar o rechazar la respuesta.

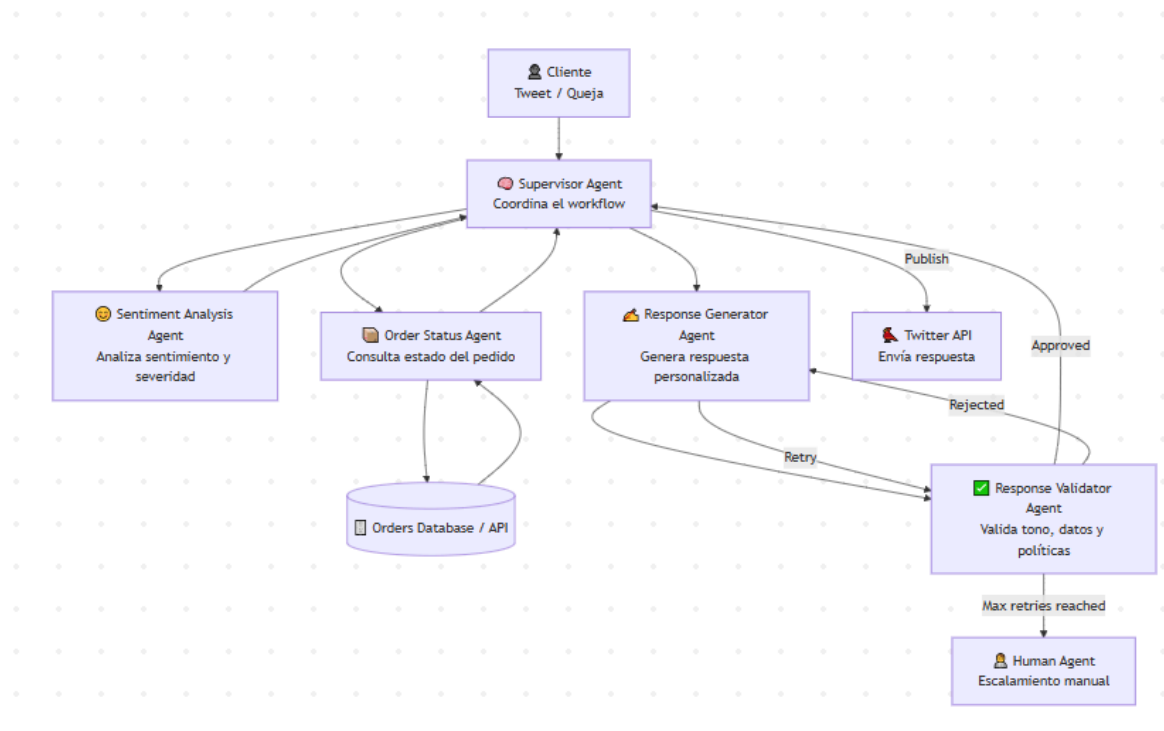
Tools:

- LLM con structured output.
- Reglas de negocio.
- Filtros de seguridad.

Podría devolver:

```
{  
  "approved": true,  
  "reason": null  
}
```

3. Diagrama de flujo lógico



Paso a paso

1. Recepción del tweet

El sistema recibe un mensaje como:

"Mi pedido tenía que llegar ayer y todavía no tengo ninguna noticia. Necesito una solución."

El Supervisor almacena el mensaje en el estado del workflow.

2. Análisis de sentimiento

El Supervisor delega el análisis al **Sentiment Analysis Agent**.

El agente puede devolver:

```
{  
  "sentiment": "negative",  
  "severity": "high"  
}
```

3. Consulta del pedido

El Supervisor llama al **Order Status Agent**, que consulta la base de datos o API del e-commerce.

Obtiene, por ejemplo:

```
{  
  "status": "delayed",  
  "expected_delivery": "2026-08-20"  
}
```

4. Generación de respuesta

Con toda la información disponible, el Supervisor llama al **Response Generator Agent**.

El agente recibe un contexto similar a:

Tweet: cliente reclama pedido atrasado
Sentiment: negative
Severity: high
Order status: delayed
Expected delivery: 2026-08-20

y genera una respuesta personalizada.

5. Validación

Antes de publicar el mensaje, el **Response Validator Agent** verifica que la respuesta sea correcta.

Si la respuesta es aprobada:

Validator → Supervisor → Twitter API

Si es rechazada:

Validator → Supervisor → Response Generator

El Supervisor puede solicitar una nueva versión.

Para evitar ciclos infinitos, se establecería un límite, por ejemplo:

MAX_RETRIES = 3

Si después de tres intentos la respuesta continúa siendo rechazada, el caso podría derivarse a un operador humano.

4. Estado compartido

En LangGraph podría definirse un estado similar a:

```
class CustomerSupportState(TypedDict):
    tweet: str
    customer_id: str | None

    sentiment: str | None
    severity: str | None

    order_id: str | None
    order_status: str | None
    expected_delivery: str | None

    generated_response: str | None
    approved: bool | None

    retry_count: int
```

Cada agente modifica solamente la parte del estado relacionada con su responsabilidad.

Esto evita enviar todo el contexto a todos los agentes y reduce tanto el ruido como el consumo de tokens.

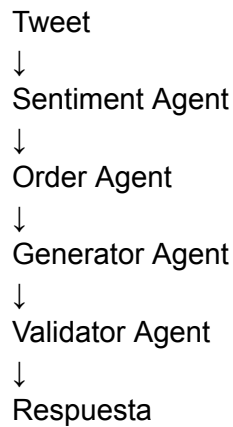
5. Justificación de la topología

Para este sistema elegiría una **topología jerárquica** porque el proceso de atención al cliente sigue un workflow relativamente predecible:

Analizar → Consultar → Generar → Validar → Responder

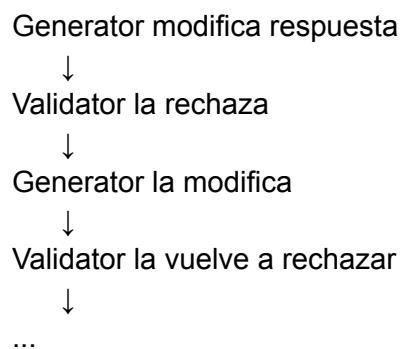
Además, existe información sensible y operacional involucrada, como estados de pedidos y datos de clientes. Por este motivo es conveniente tener un punto central que controle qué agentes pueden acceder a cada información y qué respuesta finalmente se publica.

La arquitectura jerárquica también facilita la **trazabilidad**. Ante una respuesta incorrecta es posible reconstruir fácilmente el proceso:



Una arquitectura cooperativa sería menos conveniente porque permitir que los agentes reaccionen libremente sobre un estado compartido podría producir ejecuciones innecesarias, conflictos entre agentes o ciclos de corrección difíciles de controlar.

Por ejemplo:



El Supervisor permite controlar explícitamente este comportamiento mediante límites de reintentos y reglas de escalamiento.

Conclusión

La arquitectura propuesta utiliza un **Supervisor Agent como orquestador** y varios agentes especializados para separar responsabilidades.

Supervisor:

- Sentiment Agent
- Order Status Agent
- Response Generator Agent
- Response Validator Agent

Esta topología proporciona **control, modularidad y trazabilidad**, características especialmente importantes en un sistema automatizado de atención al cliente.

Además, su estructura puede implementarse de forma natural con **LangGraph**, utilizando cada agente como un nodo, un estado compartido para almacenar los resultados y conditional edges para controlar validaciones, reintentos y escalamiento a atención humana.